

Numeric Precision Standard: IEEE 754 vs. Exact Decimal

Redacted Reference Edition -- internal file/module names replaced with fictitious placeholders

Status

Standing technical specification. Documentation of an existing, already-shipped engineering convention -- this spec does not itself change any code.

Directive (verbatim)

"Scribe our IEEE-754 rating into our latest business plan build and software tech specs."

The real rating

This app's scientific-calculator engines (physics, matrix/linear-algebra, color science, tone-color -- the "AI Engineer Calculations" tier) run in IEEE 754 double precision (binary64) -- Python's native float and Swift's native Double, the same 64-bit format both languages use by default. Verified live against Python's own `sys.float_info`, not recalled from memory:

Property	Value	Meaning
Mantissa width	53 bits (52 stored + 1 implicit)	The real bit budget for significant digits
Guaranteed decimal digits	15	Any decimal number with <=15 significant digits round-trips exactly
Machine epsilon	2.220446049250313e-16	The smallest relative gap between two representable numbers near 1
Max value	1.7976931348623157e+308	
Min positive normal value	2.2250738585072014e-308	

In practice this gives ~15-17 significant decimal digits of usable precision per number (15 guaranteed exactly; up to 17 needed to uniquely round-trip an arbitrary double). For comparison, a real, documented graphing calculator (the Texas Instruments TI-84 Plus/CE line) runs a fixed-point BCD format at 14 significant digits internally, 10 displayed -- this app's double-precision tier is a modest step above that ceiling, not a different category by itself.

Where this is the standard (and why)

Every function in `mechanics_domain.py` / `MechanicsEngine.swift`, `matrix_domain.py` / `MatrixEngine.swift`, `chroma_domain.py` / `ChromaEngine.swift`, and `tone_color_domain.py` / `ToneColorEngine.swift` computes in this format, by explicit design choice stated in each file's own documentation: "this is general math, not money." Every one of these domains ships a golden-vectors parity harness -- one language side is the reference computation, the other side is cross-checked against it to a stated numeric tolerance (typically 1e-6 to 1e-9, loosened only where a cube-root/cube pipeline genuinely accumulates more float error, and always stated explicitly, never silently).

Where this is explicitly NOT the standard, and why

Every money-related figure in this app -- budgets, balances, location-based cost-reference dollar amounts, compute-worth calculations -- uses Python's `Decimal` type under an explicit high-precision arithmetic context, never `float/Double`. This is a real, load-bearing distinction, not a style preference: IEEE 754 binary floating point cannot exactly represent most decimal fractions (the textbook example: $0.1 + 0.2 \neq 0.3$ in double precision, because neither 0.1 nor 0.2 has an exact binary representation). Money math needs EXACT decimal arithmetic -- a budget that's off by a machine-epsilon-scale rounding error is a real defect, not a rounding curiosity -- so this codebase deliberately keeps the two numeric domains (general/scientific math vs. money) on two different, purpose-fit representations rather than using one format everywhere.

A live example of why verification matters more than digit count

While building the tone-color calculator feature in this same session, a first, from-memory recollection of a published color-science matrix (a well-known perceptual color-space transform) was transcribed incorrectly. The error was caught immediately by a real property check -- multiplying the matrix by its claimed inverse should yield the identity matrix, and it did not -- before the wrong constant shipped anywhere. The corrected matrix, re-fetched verbatim from the primary published source, passed the same check. This is the practical argument for this spec existing at all: precision (15-17 digits, or the 100 digits an arbitrary-precision library demonstrated for a complex-exponent calculator concept) only produces a more accurate answer when the underlying constants and formulas feeding it are independently verified -- otherwise it is just a wrong answer stated with more confidence.

Arbitrary precision (not yet a standing engine tier)

An arbitrary-precision math library (already available in this project's Python environment) was used ad hoc this session to compute a real transcendental identity to 100 verified digits for a complex-exponent calculator concept. This is a real capability, honestly distinct from the standing double-precision tier above: it has not been wired into any shipped calculator function, and doing so (choosing where arbitrary precision is worth its real compute-time cost vs. where double precision already suffices) is a real, open design decision, not yet made.

Redaction note

This edition replaces every internal Swift/Python source-file name from the original working spec with a fictitious placeholder (e.g. `mechanics_domain.py` / `MechanicsEngine.swift` stand in for this project's real physics-calculator source files) so it can be shared outside the engineering team without exposing the actual codebase layout. All technical claims, numbers, and citations are otherwise unchanged from the working internal specification.